APPROACH DOCUMENT

Cymonic AI Hackathon

Amal Varghese

1. Executive Summary

Expense Tracker is an intelligent, AI-driven hybrid mobile application and backend ecosystem designed to automate the traditionally manual and error-prone process of corporate expense auditing.

2. Problem Statement

Managing and auditing travel expenses is a tedious and manual process for both employees and accounting teams. Employees often struggle to keep track of complex company spending policies or track their receipts properly, while auditors spend excessive hours manually cross-referencing paper or digital receipts for authenticity, correct amounts, and policy adherence.

3. Proposed Solution

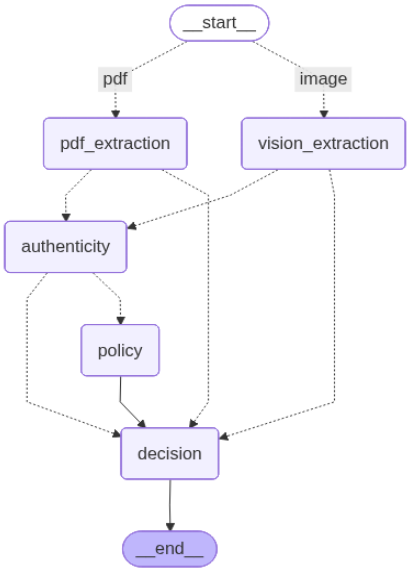
The Expense Tracker is a hybrid mobile application and intelligent backend system that automates the expense reporting and auditing process. Employees can snap photos of their receipts or upload them to the app, which automatically synchronizes with a cloud database. An AI-powered auditor uses an advanced LangGraph agent and vision models to parse the receipts, verify their authenticity, detect potential fraud, and instantly evaluate them against company policies. This allows for near real-time automated approvals or itemized feedback for policy violations, reducing the auditor's workload and enhancing visibility into company spending.

4. System Architecture

The Macrosoft Expense Tracker is a sophisticated distributed system designed to automate the lifecycle of professional travel expenses. It combines a mobile-first user experience with an agentic AI backend that performs multi-stage auditing.

The system is divided into 4 layers: React Native Frontend, FastAPI Backend, Langgraph pipeline and cloud database.

- React Native Expo Frontend: Provides easy way to make android applications using typescript and react-like structures. Easy to display and debug using expo go in phone.
- FastAPI backend: Provides routes for checking the server health, to chat with the pdf through RAG and process the receipts
- Langgraph Pipeline:



- At first the system will determine the file type and send go to appropriate node. If the input is an image to goes to vision_extraction node otherwise to pdf_extraction node.
- After extraction is completed if the input is not readable, we will move to decision node otherwise the system will determine the merchant's name, location, date etc and move to authenticity checking node
- The authenticity node will make a fraud detection in the receipt by checking date window, merchant authenticity and currency. If it is found not authentic, we move to decision node or else the system goes to policy node.
- In policy node we will make a RAG search in the chromaDB to check if the receipt violates company policy
- Finally, the decision node produces final decision with all the previous information from nodes before.

- Cloud Database: Uses Supabase for storing user data and other trip details.

5. Technical Stack

Component	Technology
Orchestration	Langgraph, Langchain
Frontend	React Native Expo, Nativewind
Backend	FastAPI, Py
Database	Postgress(supabase), ChromaDB(vector)
LLM	gemini-2.5-flash, gemini-3-pro-preview, gpy-5.4-nano
Embedding	text-embedding-3-small from OpenAI
Search Engine	Tavily Search

6. Future Scope

- Improve the speed of the system by using better models and parallel nodes.
- Migrate the frontend to native android application for better performance
- Improve the fraud detection in authenticity node with duplicate detection and improve relevancy checking
- Migrate the vector database to Pinecode to reduce latency for chatting with pdf.